



U1

USO DE LAYOUTS: LOCALIZACIÓN Y ALINEAMIENTO DE LOS COMPONENTES

0. Recordatorio. ¿Contenedor?

¿Qué es una clase de contenedor?

Las clases de contenedor son clases que pueden tener otros componentes. Entonces, para crear una GUI, necesitamos al menos un objeto contenedor.

Hay 3 tipos de contenedores.

- **Panel:** es un contenedor puro y no es una ventana en sí misma. El único propósito de un Panel es organizar los componentes en una ventana.
- **Marco:** es una ventana en pleno funcionamiento con su título e iconos.
- **Diálogo:** se puede considerar como una ventana emergente que aparece cuando se debe mostrar un mensaje. No es una ventana completamente funcional como el Marco.

Contenedores de alto nivel:

- **JFrame:** Habitualmente la clase JFrame se emplea para crear la ventana principal de una aplicación en Swing.
- **JDialog:** Ventanas de interacción con el usuario.

Contenedores intermedios:

- **JPanel:** Agrupa a otros componentes.
- **JScrollPane:** Incluye barras de desplazamiento.



0. Layout manager BorderLayout o NullLayout

Los componentes **no se recolocan ni redimensionan automáticamente**: mantienen **su posición y tamaño fijos**, sin importar cómo cambia el tamaño del contenedor.



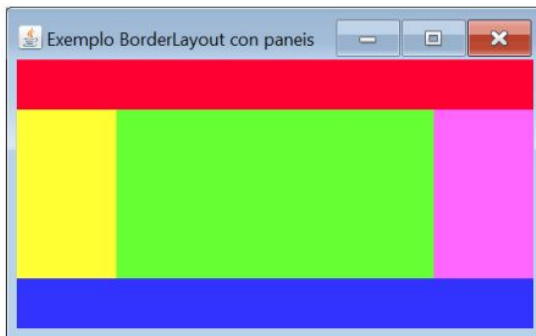
No es recomendable usar un gestor de distribución que no organice bien los componentes. Aunque la ventana no sea redimensionable, el aspecto puede variar entre sistemas y *look & feel*. Siempre conviene usar un layout adecuado.

Problemas de no usar layout

- Componentes no se adaptan al redimensionar.
- Interfaz inconsistente entre plataformas.
- Difícil de mantener y modificar.
- Riesgo de solapamiento con textos o fuentes diferentes.

1.1. Layout manager BorderLayout

El layout manager **BorderLayout** se define mediante la clase `java.awt.BorderLayout`. Su función es **dividir el contenedor en cinco zonas principales**, aunque no es obligatorio usar todas ellas. Estas zonas son:

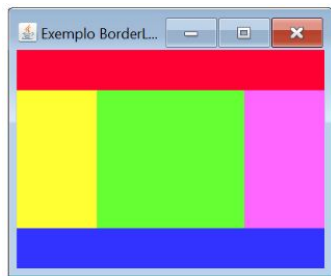
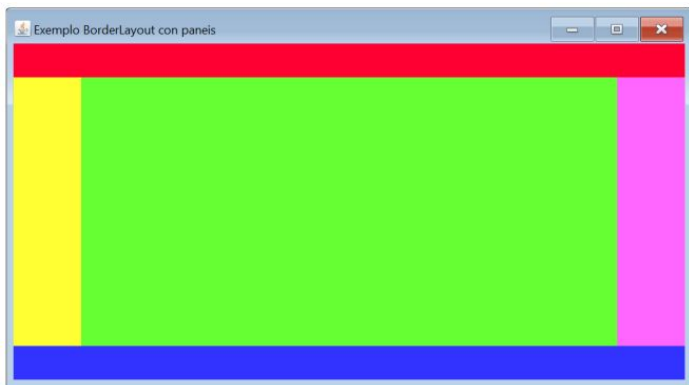


- North (Norte) → panel rojo
- South (Sur) → panel azul
- West (Oeste) → panel amarillo
- East (Este) → panel rosa
- Center (Centro) → panel verde

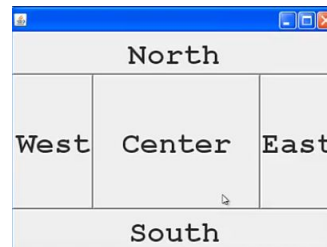
Es posible colocar **uno o varios controles** (como botones o etiquetas) directamente en cada zona, pero usar **paneles** resulta más práctico, ya que facilita la comprensión de la estructura, permite aplicar otros layout managers dentro de cada panel

1.1. Layout manager BorderLayout

La **ventaja principal** de utilizar **BorderLayout** se aprecia al **redimensionar la ventana o el formulario**



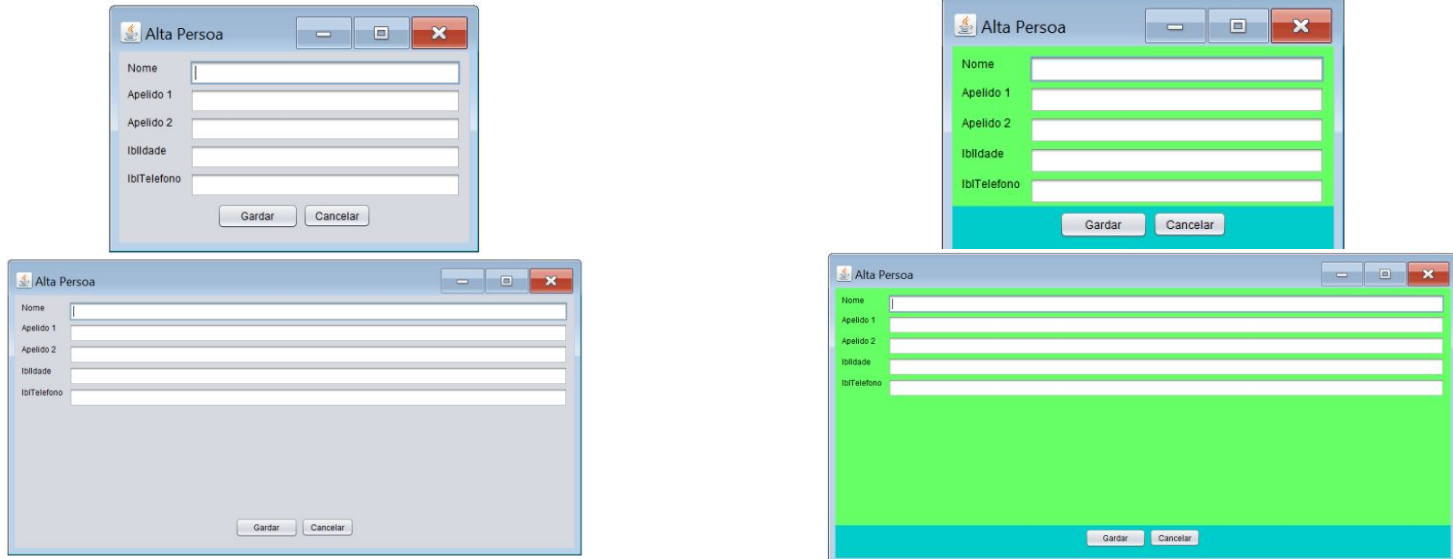
- Los paneles **rojo** y **azul** mantienen su tamaño **vertical**, mientras que **varían su tamaño horizontal**.
- Los paneles **amarillo** y **rosa** mantienen su tamaño **horizontal**, pero **cambian su tamaño vertical**.
- El panel **verde**, el **central**, **aumenta su espacio tanto horizontal como verticalmente** cuando se amplía la ventana, y **lo reduce** cuando la ventana se hace más pequeña.



1.1. Layout manager BorderLayout

Este tipo de **layout manager (BorderLayout)** se utiliza con frecuencia para diseñar aplicaciones que tienen **zonas fijas** (norte, sur, este y/o oeste), donde normalmente se colocan los **controles con las acciones principales** de la aplicación.

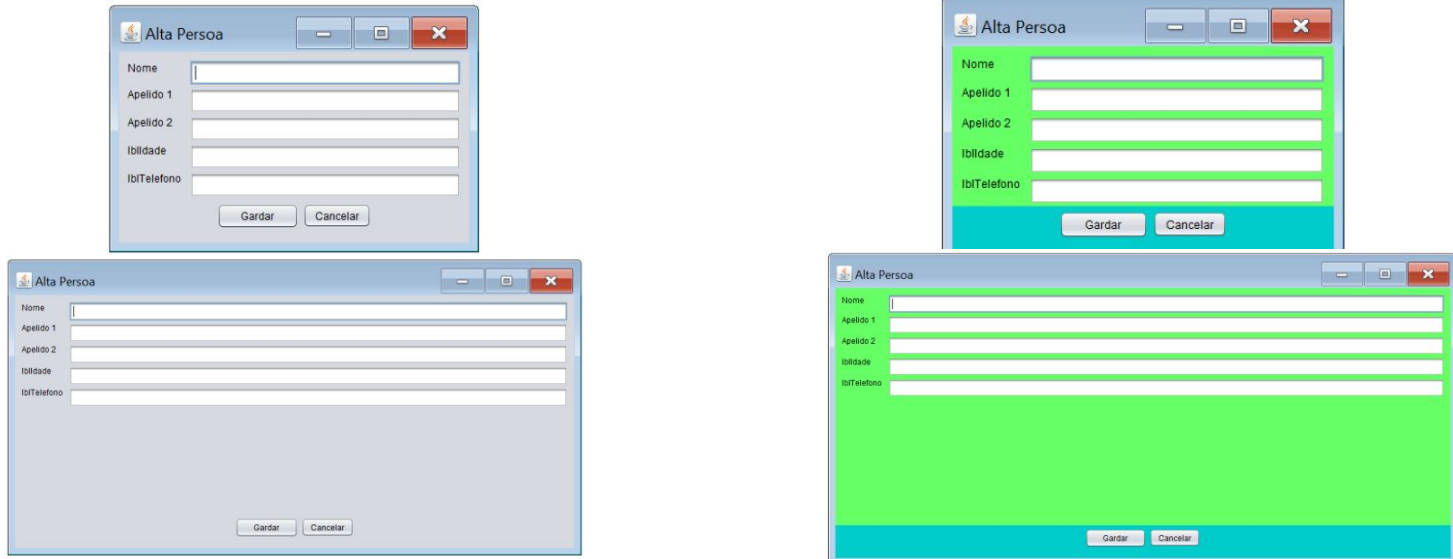
La **zona central** es donde se desarrolla realmente la funcionalidad principal del programa, y es la que **aumenta o reduce su tamaño** cuando se modifica el tamaño de la ventana o formulario.



1.1. Layout manager BorderLayout

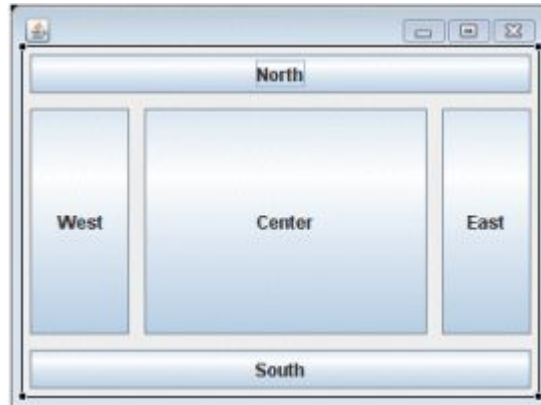
Este tipo de **layout manager (BorderLayout)** se utiliza con frecuencia para diseñar aplicaciones que tienen **zonas fijas** (norte, sur, este y/o oeste), donde normalmente se colocan los **controles con las acciones principales** de la aplicación.

La **zona central** es donde se desarrolla realmente la funcionalidad principal del programa, y es la que **aumenta o reduce su tamaño** cuando se modifica el tamaño de la ventana o formulario.



1.1. Layout manager BorderLayout

1. Usando AbsoluteLayout añade 4 botones y nombrarlos como Norte, Sur, Este y Oeste.
2. Cambia el Layout por BorderLayout. ¿Qué sucede?
3. Distribuye los elementos.

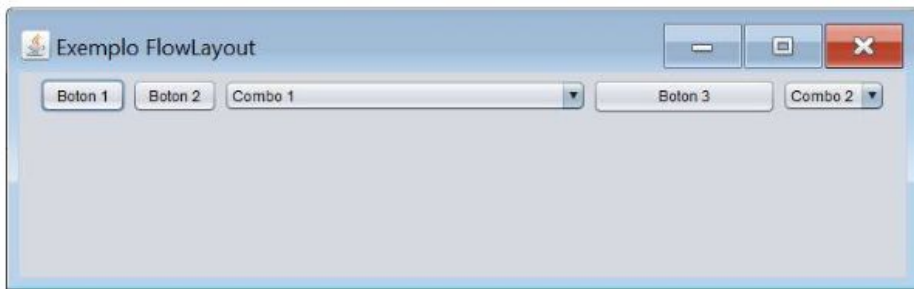


1.2. Layout manager FlowLayout

El layout manager **FlowLayout** se define mediante la clase `java.awt.FlowLayout`.

Comportamiento:

- **Coloca los componentes en una sola fila**, ajustando su tamaño según la propiedad `preferredSize` de cada uno.
- Si un componente **no cabe en la fila actual**, se **crea una nueva fila** y se coloca allí.



1.2. Layout manager FlowLayout

El layout manager **FlowLayout** se define mediante la clase `java.awt.FlowLayout`.

Configuración:

- Configurar la propiedad **preferredSize**.

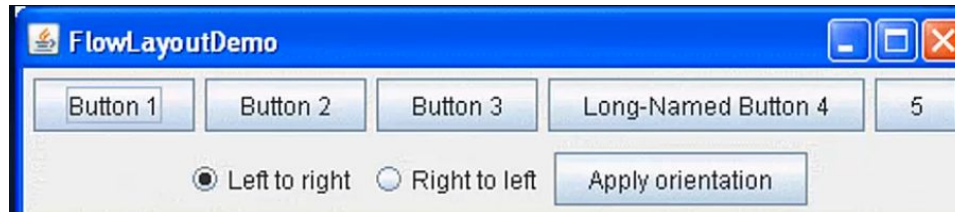
[WindowsBuilder no permite cambiar esta propiedad es necesario desde código - `.setPreferredSize(new Dimension(100, 20))`]

- Los componentes dentro de una fila pueden estar **centrados**, **alineados a la izquierda** o **alineados a la derecha**. Además, es posible **configurar la separación** entre los componentes tanto **horizontal** como **verticalmente**.



1.2. Layout manager FlowLayout

EJERCICIO: Demostración FlowLayoutDemo con la posibilidad de cambiar la orientación con la que se distribuyen los elementos.



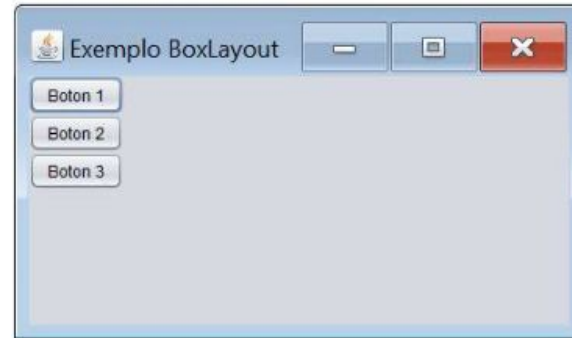
OJO! Después de cambiar el layout necesitas que el contenedor vuelva a aplicar el layout y repintarse:
`contentPane.revalidate();`
`contentPane.repaint();`

1.3. Layout manager BorderLayout

El layout manager **BoxLayout** se define mediante `javax.swing.BoxLayout`.

Comportamiento:

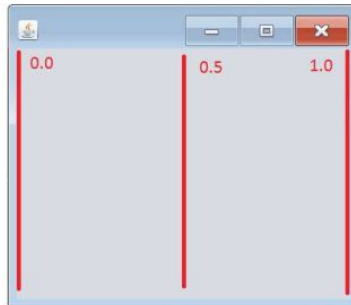
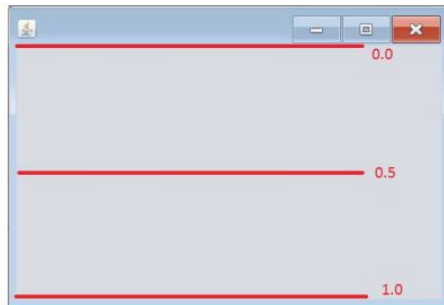
Cuando aplicamos un **BoxLayout** a un contenedor, sus componentes se **colocan uno tras otro** si el **BoxLayout** se define como **horizontal**, o se **apilan uno encima de otro** si se define como **vertical**.



1.3. Layout manager BorderLayout

Configuración:

- Se queremos mostrar los componentes de forma vertical o horizontal. Propiedad **Axis**.
- Tamaño de los componentes. Propiedades **maximumSize**, **minimumSize** e **preferredSize**
- Propiedades **alignmentY** y **alignmentX**. Alineamiento con respecto al contenedor.



- ¿Cómo separar componentes?

Strut

Horizontal o Vertical

Espaciador

☐ No

☒ Sí

Mantiene un espacio **fijo** entre componentes

Glue

Horizontal o Vertical

Espaciador

☒ Sí

☐ No

Crea espacio **elástico**, empuja los componentes hacia los extremos

1.4. Layout manager GridLayout

El layout manager GridLayout se define mediante la clase `java.awt.GridLayout`.

Su aspecto visual consiste en una rejilla (grid) formada por filas y columnas, donde cada celda tiene el mismo tamaño y cada componente ocupa una celda de esa cuadrícula.



1.4. Layout manager GridLayout

Configuración:

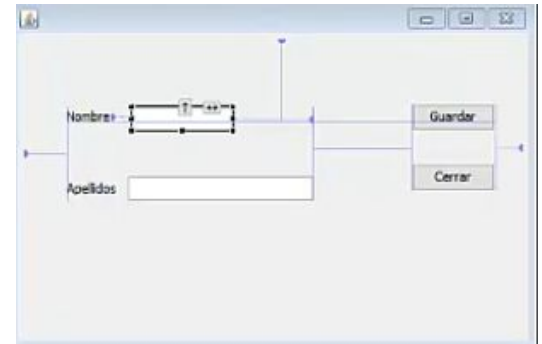
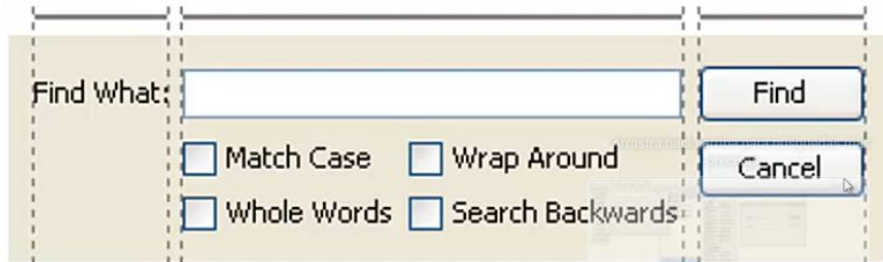
Propiedad	Descripción
rows (filas)	Número de filas en la cuadrícula. Si se pone en 0, el número de filas se ajusta automáticamente según las columnas.
cols (columns)	Número de columnas en la cuadrícula. Si se pone en 0, se ajusta automáticamente según las filas.
hgap (horizontal gap)	Espacio horizontal entre columnas (en píxeles).
vgap (vertical gap)	Espacio vertical entre filas (en píxeles).



1.5. Layout manager GroupLayout

El layout manager GroupLayout se define mediante la clase `java.awt.GroupLayout`.

Se reparten los objetos en función unos de otros.



1.6. Layout manager GridBagLayout

El layout manager GridLayout se define mediante la clase `java.awt.GridBagLayout`.

El gestor de distribución **GridBagLayout** es el layout manager que nos permite crear **distribuciones de componentes más complejas**. Es, con diferencia, el gestor más completo y flexible. Sin embargo, esta mayor funcionalidad implica **una mayor complejidad de uso**.

GridBagLayout es una **versión ampliada de GridLayout**. Diferencia?

- En GridLayout el espacio se divide en una **rejilla de celdas de tamaño similar**.
- En GridBagLayout, cada celda puede tener **diferente tamaño** y un componente puede **extenderse a varias celdas contiguas**, tanto horizontal como verticalmente.

Exemplo GridbagLayout

Nome Apellido 1 Apellido 2

Observacions

Idade

Tlf. Fijo

Tlf. Mobil

Exemplo GridbagLayout

Nome Apellido 1 Apellido 2

Observacions

Idade

Tlf. Fijo

Tlf. Mobil

1.6. Layout manager GridBagLayout

Exemplo GridbagLayout

Nome Apelido 1 Apelido 2

Observacions

Idade

Tlf. Fixo

Tlf. Mobil

Exemplo GridbagLayout

Nome Apelido 1 Apelido 2

Observacions

Idade

Tlf. Fixo

Tlf. Mobil

1.6. Layout manager GridBagLayout



Limitaciones y consideraciones prácticas de GridBagLayout

- **Complejidad:** Configurar cada componente puede ser tedioso y propenso a errores, sobre todo en formularios grandes.
- **VisualBuilder / Eclipse:** Aunque VisualBuilder permite diseñar interfaces gráficas, GridBagLayout **requiere ajustar manualmente muchas propiedades** (`gridx`, `gridy`, `weightx`, `weighty`), haciéndolo más difícil de manejar que layouts más simples como `FlowLayout` o `BoxLayout`.
- **No siempre necesario:** Para interfaces sencillas o medianas, layouts más simples son **más rápidos de configurar y mantener**.
- **Rendimiento y legibilidad:** Interfaces muy complejas con GridBagLayout pueden ser difíciles de mantener y complicar la legibilidad del código, especialmente en proyectos colaborativos.

Conclusión

GridBagLayout es **ideal para diseños complejos y precisos**, pero **no conviene usarlo para todas las interfaces**, sobre todo si se busca rapidez, simplicidad y facilidad de mantenimiento.

2. Máscaras - Look & feel

Las **máscaras** o **Look & Feel (L&F)** permiten **cambiar el aspecto visual** de las aplicaciones gráficas en Java Swing sin modificar su funcionamiento.

Un *Look & Feel* está formado por un conjunto de **clases que definen el estilo visual** de cada componente de la interfaz (botones, menús, cuadros de texto, etc.).

Cuando aplicamos un *Look & Feel* a una aplicación, le indicamos a Swing que utilice las **definiciones gráficas** de ese estilo para dibujar los componentes.



2. Máscaras - Look & feel

Look & Feel incluidos en Java

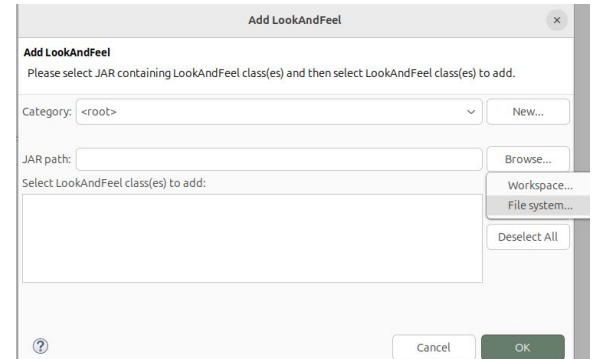
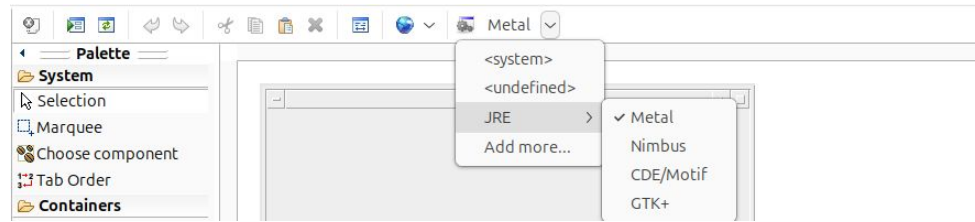
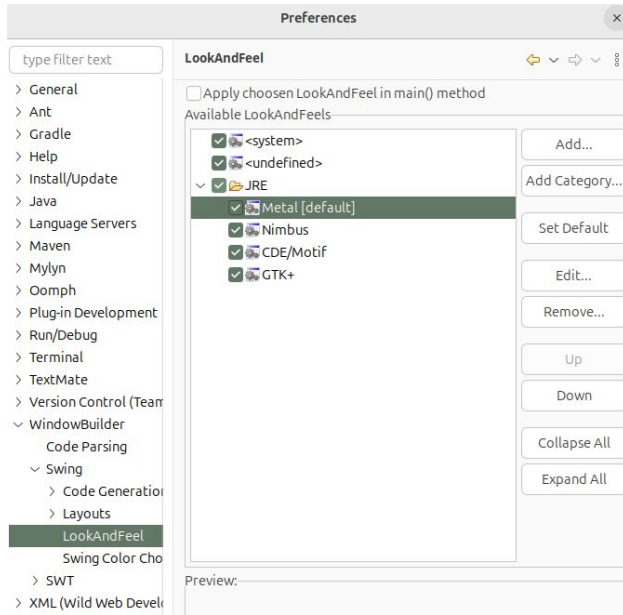
Java incluye varios estilos preinstalados que se pueden activar desde el código o el entorno de desarrollo

Nombre	Clase asociada
Metal	<code>javax.swing.plaf.metal.MetalLookAndFeel</code>
Nimbus	<code>javax.swing.plaf.nimbus.NimbusLookAndFeel</code>
Motif	<code>com.sun.java.swing.plaf.motif.MotifLookAndFeel</code>
Windows	<code>com.sun.java.swing.plaf.windows.WindowsLookAndFeel</code>
Windows Classic	<code>com.sun.java.swing.plaf.windows.WindowsClassicLookAndFeel</code>

2. Máscaras - Look & feel

Look & Feel externos. Se pueden usar otros Look & Feel desarrollados por terceros:

- **Quaqua Look & Feel** → <http://www.randelshofer.ch/quaqua/>
- **JTattoo Look & Feel** → <https://jtattoo.de/Download.html>



EJERCICIO - COMBINAR LAYOUTS

